

Automatic Zoom

Integrar un Agente Google ADK en

Microsoft Teams con Microsoft 365

Agents SDK (Python)

Objetivo: Aprovechar tu agente existente desarrollado con el Google Agent Development Kit (ADK), exponiéndolo como servicio en la nube (Google Cloud Platform) y conectándolo a Microsoft Teams mediante la capa de integración Microsoft 365 Agents SDK para Python, sin reimplementar su lógica. A continuación se presenta un análisis profundo de la documentación oficial y mejores prácticas para publicar el agente en Teams como una aplicación completa (bot de chat 1:1, en canales de Teams, etc.), cubriendo capacidades de la SDK, arquitectura, pasos de implementación, consideraciones de despliegue en GCP, detalles de configuración del manifest de Teams y posibles limitaciones/soluciones.

1. SDK multicanal,
 backend independiente
 Microsoft 365 Agents SDK
 (Python) permite crear bots
 integrados en Teams y Copilot
 en cualquier plataforma,
 reutilizando tu lógica de IA
 existente (por ejemplo, tu
 agente ADK), pues soporta
 cualquier modelo/
 orquestador externo sin
 requerir reimplementación. Le
 llamamos "agente proxy"
 porque es una capa delgada
 que solo enruta mensajes
 entre Teams y tu agente
 backend.
2. Conexión vía Azure
 Bot Service & Teams
 Para integrarse con Teams, la
 SDK se apoya en Azure Bot
 Service y el protocolo Bot
 Framework para entregar
 actividades (mensajes de
 Teams) a tu agente en GCP.
 Aunque el agente se ejecute
 en GCP, el plano de control
 sigue siendo Microsoft (Bot
 Service envía JSON con un

bearer token firmado por botframework.com). La SDK proporciona seguridad y autenticación para validar estos tokens antes de procesar los mensajes entrantes.

2 1

funciones como mencionar usuarios (p. ej., incluyendo @nombre en el texto) o anidar respuestas, ya que no hay atajos predefinidos para ello.

Dependencias con Bot Framework/Teams: Al ser una evolución del Bot Framework, la Agents SDK utiliza la infraestructura del Bot Framework (Azure Bot Service) para el flujo de mensajes con Teams 3. Esto implica que necesitas un registro de bot en Azure AD (o a través del Portal de Desarrolladores de Teams) para obtener un Microsoft App ID (ClientId) y proporcionar credenciales (secret o certificado) que la SDK usará para autenticarse ante el servicio de bot de Teams 4. Es decir, aunque la implementación se ejecute en GCP, debes configurar un recurso de Bot (App registration en Entra/Azure AD) para que Teams conozca tu bot y dirija los mensajes a tu URL de GCP 4 5. Esta dependencia es necesaria para que la comunicación entre Teams y tu agente sea segura y confiable (mediante tokens JWT firmados por Microsoft Bot Service como se indicó).

3. Arquitectura de referencia: Teams ↔ Agents SDK

↔ Backend (ADK en GCP)

A nivel lógico, la solución consta de dos componentes principales:

el Bot de Teams desarrollado con la M365 Agents SDK (que funciona como frontend y adaptador de canales)

el agente backend existente basado en Google ADK (orquestador/motor de IA) ejecutándose en GCP (nuestro servicio de IA).

La comunicación sigue el flujo típico de un bot en Teams adaptado a tu infraestructura híbrida:

-
-

Paso 1: Usuario de Teams envía un mensaje al bot

El usuario escribe una consulta o mensaje en Teams (ya sea en un chat 1:1 con el bot o en un canal mediante @mención al bot).

Paso 2: Teams envía la actividad al servicio de bot

Microsoft Teams pasa el mensaje al Azure Bot Service (canal Bot Framework), el cual encapsula el mensaje como una Activity JSON(tipo: Message) junto con un token JWT de seguridad.

Teams (disponible en la sección Apps personalizadas del administrador de Teams o directamente en el cliente Teams)【18+L67-L75】.

Publicación general: Si planeas un despliegue a toda tu organización, sigue el proceso estándar de publicación de apps internas en Teams (por ejemplo, a través del catálogo de aplicaciones corporativas) o, si fuera

multitenant, somete la app a validación de la tienda de Teams (no suele aplicar en escenarios privados).

4. Riesgos comunes, limitaciones conocidas y mejores prácticas

Autenticación y seguridad: Error común: olvidar configurar correctamente la autenticación. Si tu agente GCP no valida los tokens JWT de Microsoft, Teams rechazará la conexión. Sigue la documentación para usar AgentAuthConfiguration y jwt_authorization_middleware con las credenciales de tu App ID/Secret【23+L3-L10】. Buena práctica: manejar errores de autenticación devolviendo códigos HTTP apropiados (401/403) para facilitar el debugging.

Tiempos de respuesta: Los mensajes de Teams a bots suelen tener un timeout en torno a 15-30 segundos; si tu backend ADK tarda más, el usuario puede recibir un error de bot no respondiente. Mitigación: Intenta que las operaciones de IA sean eficientes. Si tu respuesta es muy larga o compleja, aprovecha la capacidad de respuestas en streaming de la Agents SDK【11+L34-L41】: envía fragmentos parciales conforme se van generando, para que el usuario vea progresivamente la respuesta en lugar de esperar todo de una vez. También puedes utilizar indicadores “escribiendo” (typing) al inicio de la interacción para mejorar la experiencia【11+L34-L40】 (mostrando que el bot está procesando la solicitud larga).

Manejo de contexto y estado: Si tu agente ADK mantiene contexto, asegúrate de identificar correctamente la conversación (especialmente en canales; ver punto 5). Error común: mezclar contextos de usuarios diferentes. Puedes usar el ID de conversación incluido en cada Activity de Teams para separar sesiones por canal/usuario. También considera los límites de tamaño de mensajes en Teams (actualmente ~28 KB por mensaje). Si la respuesta de tu IA es muy extensa, podrías resumirla o dividirla en múltiples mensajes para evitar truncamiento.

•

Manifest y capacidades: Verifica que el manifest de Teams contenga todos los apartados obligatorios (identificadores correctos, iconos, descripciones) y que la sección de permission & validDomains esté acorde a tus necesidades. No olvides incluir tu dominio GCP en “validDomains” del manifest para que Teams permita las conexiones a él.

Prácticas adicionales:

Implementa un comando de ayuda (/help) que presente instrucciones básicas al usuario la primera vez (véase el <https://learn.microsoft.com/es-es/microsoft-365/agents-sdk/quickstart?tabs=python#additional-sample-code> que envía un mensaje de bienvenida en el evento membersAdded) 【1+L150-L158】. Esto mejora la UX inicial.

Registra los errores y excepciones en tu servicio GCP para facilitar la corrección de fallos. Por ejemplo, loguea respuestas vacías o errores de conexión al backend ADK, y considera responder al usuario con un mensaje de disculpa y sugerir reintento si algo falla.

Prueba en diferentes contextos: chat 1:1, chat grupal, canal de Teams, para asegurarte que la experiencia es consistente y que el agente maneja bien los distintos escenarios (ejemplo: múltiples usuarios en canal).

Mantente atento a la documentación actualizada: la Agents SDK y Teams Platform evolucionan rápidamente (ej. Teams SDK vs Agents SDK convergiendo en capacidades【11+L15-L24】). Consulta periódicamente las notas de los SDK updates para conocer nuevas características o cambios.

5. Referencias a documentación oficial y ejemplos relevantes

Quickstart de Microsoft 365 Agents SDK (Python): guía paso a paso para crear un agente básico (Echo) en Python【1+L28-L36】【1+L129-L137】.

Este recurso muestra cómo inicializar un proyecto (pip install ...), implementar la clase AgentApplication y definir handlers de eventos (como AGENT_APP.activity("message"))【1+L143-L151】, además del arranque del servidor AIOHTTP en /api/messages 【1+L92-L99】. Es la base para estructurar tu agente proxy.

Documentación de Microsoft 365 Copilot/Agents (Custom Engine Agents): describe la filosofía de agentes con motor personalizado y da un diagrama de flujo de integración de un backend ajeno (llamado “Custom Engine

-
-
-
-
-
-

Agent”)【10+L63-L71】 – casi idéntico a tu escenario. Muestra cómo un “Proxy Agent” hecho con M365 Agents SDK recibe mensajes de Copilot (o Teams), los reenvía a un backend IA (por HTTP REST) y retorna la respuesta al ecosistema Microsoft【10+L67-L75】.

Comparativa de SDKs para Teams: Pick the right SDK for your Teams agent en la doc de Teams puntualiza las diferencias entre la M365 Agents SDK (enfoque multicanal) y la Teams SDK (enfoque profundo en Teams).

Resume las capacidades disponibles con Agents SDK (mensajes, typing, streaming, actualización de conversaciones, OAuth, etc.) frente a las adicionales del Teams SDK【11+L32-L40】【11+L50-L58】. Es útil para comprender qué esperar (y qué no) de la Agents SDK en Teams.

Guía de despliegue en Azure y Teams: Indica cómo registrar tu bot en Azure (obtener AppID/clave), configurar el endpoint /api/messages con tu dominio GCP en la configuración del Bot Service【18+L37-L45】, preparar manifest.json (reemplazar AppID y dominio de bot), e instalar la app en Teams【18+L61-L69】. También cubre la carga de un manifest personalizado en un tenant (Teams Admin Center) para pruebas【18+L67-L75】.

Documentación Microsoft (español) – Agente 365 en GCP: Recurso específico en español sobre desplegar un agente en Cloud Run【22+L229-

L237】. Detalla la creación de la configuración (a365.config.json con messagingEndpoint apuntando a GCP【22+L229-L238】【22+L231-L239】) y resalta que tras desplegar el código, hay que crear la identidad del agente en Entra ID, registrar el endpoint de Bot Framework y publicar en Teams 【22+L258-L266】. Confirma también la necesidad de habilitar la validación JWT incluso en GCP (muestra código Node con authorizeJWT , señalando que “esto es requerido incluso en GCP – el control plane sigue siendo Microsoft”)【23+L3-L10】.

Repositorio de ejemplo – M365 Custom Engine Agent: Un ejemplo completo en Python disponible en GitHub muestra un proxy agent (M365 Agents SDK) que se integra con un backend IA externo para Copilot/Teams 【10+L32-L40】. Incluye código fuente (en carpeta cea-proxy-py), un diagram de secuencia de la integración【10+L67-L75】 , configuración de manifest (appPackage/manifest.json) y scripts de despliegue. Puede servirte de referencia práctica para diseñar tu agente proxy.

-
-
-
-

En resumen, aprovechar el Microsoft 365 Agents SDK para integrar tu agente Google ADK en Teams es viable y está soportado: la SDK facilita la conexión con Teams y Copilot sin que tengas que migrar la lógica de tu agente. Solo debes encargarte de la configuración de credenciales (App ID/Secret de Azure AD), exponer tu servicio en GCP con https, y empaquetar la app de Teams correctamente【18+L61-L69】. Con estos pasos, tu agente podrá brindar sus funcionalidades de IA a los usuarios de Microsoft Teams, manteniendo tu infraestructura en Google Cloud y aprovechando lo mejor de ambos ecosistemas. ¡Buena suerte con tu integración! ☐☐